Pointwise-Optimal Computational Complexity: Sorting under Partial Information against Hidden Completion

Yunbei Xu
National University of Singapore
yunbei@nus.edu.sg

Abstract

Worst-case complexity can obscure the structure already revealed by an instance. Sorting under partial information gives a clean finite model in which the pointwise refinement is exact. Given a public partial order P, the hidden object is an unknown linear extension, and the optimal worst-case number of additional comparisons is $\Theta(\log_2 e(P))$, where $e(P)$ is the number of compatible extensions. Existing results show that a single global comparison rule matches, for every public partial order P, the clairvoyant benchmark that knows P but not the hidden linear extension. We interpret this guarantee as an explicit Bellman log-potential argument, attaining an abstract notion of pointwise worst-case optimality in the structured resource model. We then pass from finite state spaces to uncountable numerical keys by revealing only finite-resolution bucket labels. The resulting residual profile has an explicit merge-sort potential inside the buckets. This extension is the sorting analogue of the finite-scale reduction used in pointwise statistical dimension, while remaining a deterministic comparison theorem. The note uses the solved example of sorting under partial information to illustrate pointwise worst-case complexity in deterministic computation and to connect it with recent notions of pointwise and algorithmic complexity in learning.

1 Introduction

Worst-case analysis asks for one number. For comparison sorting on n items, the answer is $\Theta(n \log n)$. The answer is correct, but it is too compressed: it gives the same label to an entirely unordered list and to a list whose order is almost already known.

Sorting with partial information is the simplest place where this compression can be undone. The algorithm is given a partial order P on the items. The hidden truth is an arbitrary total order extending P. The algorithm may ask additional comparisons and must recover the hidden total order. If

$$e(P) := |\operatorname{Lin}(P)|$$

denotes the number of linear extensions of P, then $\log_2 e(P)$ is the number of remaining binary decisions. The theorem says that this is not only an information lower bound. It is the correct pointwise comparison complexity, up to a universal constant.

This note is precise about what is new. The comparison lower bound is the classical decision-tree counting bound. The asymptotically matching upper bound belongs to the long theory of sorting under partial information (Fredman, 1976; Kahn and Saks, 1984; Kahn and Kim, 1995; Cardinal et al., 2013; Haeupler et al., 2025; Rutschmann, 2026). The contribution here is to formulate this theory as an exact benchmark for pointwise worst-case computation and to make the Bellman

potential structure explicit. The public point is P; the hidden states are $\mathrm{Lin}(P)$; the resource is the number of additional comparisons; and the pointwise profile is

$$d_{\mathrm{sort}}(P) = \log_2 |\mathrm{Lin}(P)|.$$

The broader lesson is about information revelation. If the information revealed to the learner or algorithm is formulated as the instance, then one can take a worst case over the hidden completions that remain. The benchmark may be clairvoyant about the public instance P: it may choose the best decision tree after seeing P, but it still must work for every hidden extension in $\mathrm{Lin}(P)$. The theorem below shows universal instance-optimality in this sense. A single globally valid comparison rule matches, for every P, the same order of comparisons as this instance-wise clairvoyant benchmark.

The central proof mechanism is a logarithmic Bellman potential. We use "Bellman" in the elementary dynamic-programming sense of a resource-to-go function: after each admissible answer, the potential must fall by at least the cost of the query. This connects the note to the Bellman-sufficient viewpoint on complexity (Xu, 2026b), but here no abstract existence principle is invoked; the potential is written down explicitly. A theorem of Kahn and Saks gives a universal constant $\beta < 1$ such that every non-total poset admits an incomparable pair whose two possible comparison outcomes each leave at most a β-fraction of the linear extensions. With their constant one may take $\beta = 8/11$. Therefore

$$\Phi(P) := \frac{\log_2 e(P)}{\log_2(11/8)}$$

drops by at least one after the chosen comparison, no matter which answer the hidden extension gives. This proves the universal comparison upper bound. It is not a computationally efficient way to find the pair, because exact extension counts are hard; efficient implementations use deeper algorithms. But for comparison complexity, the potential is explicit and the quantifiers are transparent.

The finite theorem also points beyond finite state spaces. If the keys are real numbers in $[0, 1)$, an exact input has uncountably many possible values. But a practical sorting system often first sees a finite-resolution observation, such as the first b bits of each key. Those bits create ordered buckets. If bucket a contains m_a keys, then the remaining exact sorting cost is

$$\Theta\left(\sum_a \log_2(m_a!)\right).$$

This is the sorting analogue of the finite-scale move in pointwise statistical dimension (Li and Xu, 2026): the relevant object is not the singleton real input, but the finite-resolution cell that remains after coarse information has been revealed. The analogy is about localization and scale, not about the resource being measured; here the resource is still additional comparisons.

There is also a useful conceptual warning. In the finite comparison model, $\log e(P)$ is close to an Occam length: after conditioning on P, there are exactly $e(P)$ possible hidden objects. In uncountable statistical models, singletons typically have prior mass zero, and pointwise statistical dimensions must use finite-resolution metric balls around the realized hypothesis. Section 7 makes this mapping precise and records the polyhedral interpretation through order polytopes.

2 Pointwise resource profiles

We use only the minimal language needed for the sorting example. Fix a computational model, a correctness requirement, and a resource measure, in the spirit of Blum's axiomatic treatment of complexity (Blum, 1967). At input size n, let $x \in \mathcal{X}_n$ denote the public structure. There may also

be hidden completions $\omega \in \Omega(x)$ consistent with x. A globally valid algorithm is one algorithm that is correct for every public structure and every compatible hidden completion. Its pointwise worst-case resource profile is

$$p_{A,n}(x) := \sup_{\omega \in \Omega(x)} \mathrm{Time}(A, x, \omega), \qquad x \in \mathcal{X}_n.$$

When there is no hidden completion, $\Omega(x)$ is a singleton and this reduces to the ordinary running-time profile.

Worst-case complexity collapses the profile to $\sup_x p_{A,n}(x)$ and then optimizes over algorithms. Pointwise complexity keeps the function $x \mapsto p_{A,n}(x)$, but it keeps the same uniformity discipline: the algorithm is fixed before the input is chosen. An auxiliary structure–a tree decomposition, active set, preconditioner, entropy vector, decision tree, or heap trace–may be important for proving or implementing an upper bound. It is not a primitive part of the pointwise complexity unless the computational model makes it part of the input; if an algorithm constructs it, its construction cost belongs to the chosen resource.

The strongest form of a pointwise theorem compares one global algorithm with the conditional optimum at each public point. For a public structure x, let $C^\star(x)$ be the least worst-case resource needed after x is fixed, allowing the decision procedure to depend on x. This benchmark knows the revealed instance but not the hidden completion. A single algorithm A is pointwise optimal up to a constant if

$$p_{A,n}(x) \le C\, C^\star(x) \quad \text{for every } x \in \mathcal{X}_n$$

with a universal constant C. We call this universal instance-optimality of the pointwise profile. The quantifier pattern is: first prove a conditional lower bound at every public point, then show that one uniform algorithm matches all those lower bounds simultaneously.

3 Sorting under partial information

Let $[n] = \{1, \dots, n\}$. A partial order P on $[n]$ is public information. A linear extension of P is a permutation π of $[n]$ such that whenever $i <_P j$, item i appears before item j in π. We write $\mathrm{Lin}(P)$ for the set of linear extensions and $e(P) = |\mathrm{Lin}(P)|$.

The hidden input is an unknown $\pi \in \mathrm{Lin}(P)$. An algorithm receives P, may adaptively query pairwise comparisons, and must output π. A query on (i, j) returns which of i and j appears first in the hidden extension. Comparisons already forced by P are public and need not be queried; the comparison cost counts only additional queries to the hidden total order.

For a deterministic algorithm A, let $q_A(P, \pi)$ be the number of additional comparisons made on hidden extension π, and define the public-point worst-case profile

$$Q_A(P) := \max_{\pi \in \mathrm{Lin}(P)} q_A(P, \pi).$$

The conditional optimum at P is

$$C^\star(P) := \inf_A Q_A(P),$$

where the infimum is over deterministic comparison decision procedures that are correct for every $\pi \in \mathrm{Lin}(P)$. Equivalently, $C^\star(P)$ is the minimum height of a comparison decision tree, allowed to be designed after P is fixed, that identifies every extension of P. The upper bounds below are stronger: they are achieved by uniform algorithms that receive P as input.

Definition 3.1 (Pointwise sorting dimension)**.** *The pointwise comparison dimension of a partial order P is*

$$d_{\mathrm{sort}}(P) := \log_2 e(P).$$

It is the number of binary comparison bits, in Shannon's logarithmic sense, that remain after the public information P has been revealed (Shannon, 1948).

4 A Bellman potential for universal partial sorting

We first record the elementary dynamic principle used in the proof. It is the finite decision analogue of a Bellman inequality.

Lemma 4.1 (Bellman potential)**.** *Consider an adaptive comparison problem whose public states are s, whose terminal states require no further query, and whose query q at state s leads to one of the successor states $s_{q,o}$. Suppose there is a nonnegative function Φ and a query choice $q(s)$ for every nonterminal state such that*

$$\Phi(s) \geq 1 + \max_o \Phi(s_{q(s),o}), \qquad \Phi(s) = 0 \text{ on terminal states.}$$

Then the algorithm that always asks $q(s)$ uses at most $\Phi(s_0)$ comparisons from the initial state s_0. If Φ is real-valued, the integer number of comparisons is at most $\lfloor \Phi(s_0) \rfloor$, and hence at most $\lceil \Phi(s_0) \rceil$.

Proof. Let $T(s)$ be the worst-case number of future comparisons made by the stated algorithm from state s. For terminal s, $T(s) = 0 \leq \Phi(s)$. For nonterminal s,

$$T(s) = 1 + \max_o T(s_{q(s),o}).$$

Induction on the number of remaining feasible hidden states gives $T(s_{q(s),o}) \leq \Phi(s_{q(s),o})$, and therefore

$$T(s) \leq 1 + \max_o \Phi(s_{q(s),o}) \leq \Phi(s).$$

Since $T(s)$ is an integer, the stated rounded bounds follow. □

For posets, the needed potential is logarithmic. If i and j are incomparable in P, write $P^{i<j}$ and $P^{j<i}$ for the transitive closures obtained by adding the corresponding relation. The two sets of extensions form a partition:

$$e(P^{i<j}) + e(P^{j<i}) = e(P).$$

Theorem 4.2 (Balanced-pair theorem, Kahn–Saks)**.** *There is a universal constant $\beta < 1$ such that every finite partial order P that is not total has an incomparable pair (i, j) satisfying*

$$e(P^{i<j}) \leq \beta e(P), \qquad e(P^{j<i}) \leq \beta e(P).$$

One may take $\beta = 8/11$ by the theorem of Kahn and Saks (1984).

The full 1/3–2/3 conjecture would allow $\beta = 2/3$, but the weaker constant is already enough for $O(\log e(P))$ comparisons. The resulting potential is explicit.

Proposition 4.3 (Logarithmic Bellman potential for partial sorting)**.** *Let* $\beta = 8/11$ *and set*

$$\kappa_{\mathrm{KS}} := \frac{1}{\log_2(1/\beta)} = \frac{1}{\log_2(11/8)}.$$

For every finite partial order P*, define*

$$\Phi_{\mathrm{KS}}(P) := \kappa_{\mathrm{KS}} \log_2 e(P).$$

At each nonterminal state, choose the lexicographically first incomparable pair, among all pairs, that satisfies the two inequalities in Theorem 4.2. Then Φ_{KS} *satisfies the Bellman inequality in Lemma 4.1. Consequently this uniform comparison rule sorts every hidden extension of* P *using at most*

$$\lceil \kappa_{\mathrm{KS}} \log_2 e(P) \rceil$$

additional comparisons.

Proof. If P is total, then $e(P) = 1$ and $\Phi_{\mathrm{KS}}(P) = 0$. Otherwise choose the lexicographically first pair (i, j) satisfying Theorem 4.2. For either outcome $P' \in \{P^{i<j}, P^{j<i}\}$,

$$\Phi_{\mathrm{KS}}(P') = \kappa_{\mathrm{KS}} \log_2 e(P') \le \kappa_{\mathrm{KS}} \log_2(\beta e(P)) = \Phi_{\mathrm{KS}}(P) + \kappa_{\mathrm{KS}} \log_2 \beta = \Phi_{\mathrm{KS}}(P) - 1.$$

Thus $\Phi_{\mathrm{KS}}(P) \ge 1 + \max\{\Phi_{\mathrm{KS}}(P^{i<j}), \Phi_{\mathrm{KS}}(P^{j<i})\}$, and Lemma 4.1 applies. □

This is a concrete Bellman log-potential dynamic program. The state is the current public poset, the action is the incomparable pair to query, and Φ_{KS} is a resource-to-go upper bound whose one-step decrease pays for the query. This is the precise sense in which the proof instantiates the Bellman-sufficient complexity viewpoint of Xu (2026b). The argument proves the comparison upper bound cleanly, but it should not be confused with an efficient implementation. Finding the balanced pair by exact counting may require hard preprocessing, since counting linear extensions is #P-complete (Brightwell and Winkler, 1991). Polynomial-time and near-linear-time sorting under partial information require the deeper algorithmic machinery developed in the cited literature. The point of Proposition 4.3 is narrower: at the level of comparison resources, the potential is explicit and its one-step decrease proves universal instance-optimality.

Theorem 4.4 (Pointwise optimality of partial-information sorting)**.** *For every finite partial order* P*,*

$$C^\star(P) \ge \lceil \log_2 e(P) \rceil.$$

Moreover, there exists a single deterministic comparison algorithm A_{bal}*, valid for every finite partial order, such that*

$$Q_{A_{\mathrm{bal}}}(P) \le \lceil \kappa_{\mathrm{KS}} \log_2 e(P) \rceil$$

for every P*, with zero comparisons when* $e(P) = 1$*. Hence*

$$C^\star(P) = \Theta(\log e(P))$$

for all P *with* $e(P) \ge 2$*, and* $C^\star(P) = 0$ *exactly when* $e(P) = 1$*.*

If the public information is given as an acyclic directed graph $G = (V, E)$ *whose reachability order is* P_G*, then known efficient implementations sort using* $O(\log e(P_G))$ *additional hidden-order comparisons. Total running time depends on the input and preprocessing convention. Under the standard DAG-input convention, topological heapsort runs in* $O(|V| + |E| + \log e(P_G))$ *time and* $O(\log e(P_G))$ *hidden-order comparisons (Haeupler et al., 2025). In the preprocessing formulation, the unified-bound-heap algorithm gives* $O(|E|)$ *preprocessing and then* $O(\log e(P_G))$ *sorting time and comparisons in that model (Rutschmann, 2026).*

Proof. The lower bound is the decision-tree counting bound, applied after conditioning on P. Fix P. A deterministic comparison procedure induces a binary decision tree on the answers that are not already forced by P; branches inconsistent with P may be deleted. Each root-to-leaf path is a sequence of additional comparison answers. At a leaf, the procedure outputs one extension of P. If two distinct extensions $\pi, \pi' \in \mathrm{Lin}(P)$ reached the same leaf, the procedure would output the same order on both hidden inputs and therefore would be wrong on at least one of them. Thus the tree must have at least $e(P)$ leaves. A binary tree of height h has at most 2^h leaves, so $h \geq \lceil \log_2 e(P) \rceil$. Taking the minimum over all decision trees for this fixed P gives $C^\star(P) \geq \lceil \log_2 e(P) \rceil$.

The comparison upper bound is Proposition 4.3. If $e(P) = 1$, the public order has a unique linear extension, so that extension can be output without querying the hidden order. Combining the lower and upper bounds proves the pointwise equivalence. The efficient total-time statements are the cited algorithmic theorems. □

The theorem is stronger than ordinary worst-case optimality. Taking the supremum over all partial orders on n items gives the usual comparison-sorting bound. Indeed, for every partial order P, the set $\mathrm{Lin}(P)$ is a subset of all $n!$ permutations, so $e(P) \leq n!$; equality holds exactly in the no-information case, where no two distinct items are publicly comparable. Thus ordinary worst-case sorting is the maximum of the pointwise profile, while the same globally valid algorithm also pays only for the remaining extension entropy at every public point P.

Equivalently, Theorem 4.4 is an instance-optimality theorem with the instance chosen correctly. The instance is not the hidden total order π, because no correct algorithm may know π in advance. The instance is the revealed partial order P. Against the clairvoyant benchmark that knows P and designs the best comparison tree for $\mathrm{Lin}(P)$, the uniform algorithm loses only a constant factor. This is the precise sense in which pointwise worst-case profiles are compatible with the usual complexity-theoretic discipline of comparing to an instance-wise optimum without allowing the solution itself to be revealed.

Corollary 4.5 (A pointwise refinement of worst-case sorting)**.** *Let $\mathcal{P}_n$ be the family of partial orders on $[n]$. There is a globally valid deterministic comparison-sorting algorithm A_{bal} whose profile satisfies*

$$Q_{A_{\mathrm{bal}}}(P) = \Theta(d_{\mathrm{sort}}(P))$$

for every $P \in \mathcal{P}_n$ with $e(P) \geq 2$, while

$$\sup_{P \in \mathcal{P}_n} Q_{A_{\mathrm{bal}}}(P) = \Theta(n \log n).$$

Thus $P \mapsto \log e(P)$ is an exact pointwise refinement of the scalar worst-case sorting complexity.

5 Examples and clarification

No information: the worst public point. An antichain is simply the partial order with no known comparison outcomes: no two distinct items are publicly comparable. Then every permutation is compatible with the public information, so $e(P) = n!$. Stirling's formula gives

$$d_{\mathrm{sort}}(P) = \log_2 n! = \Theta(n \log n).$$

Since no partial order can have more than $n!$ linear extensions, this no-information case is the public point that recovers the ordinary worst-case theorem for comparison sorting.

Complete information. If P is a chain, then $e(P) = 1$. The hidden order is already determined. The pointwise cost is zero additional comparisons, even though the input size is still n. If total running time rather than comparisons is measured, the cost of reading and outputting the public order must be charged.

Ordered blocks. Suppose the items are partitioned into blocks $B_1, \dots, B_r$, the public information says that every item in B_a precedes every item in B_b whenever $a < b$, and there is no information inside a block. Then

$$e(P) = \prod_{a=1}^{r} |B_a|!, \qquad d_{\mathrm{sort}}(P) = \sum_{a=1}^{r} \log_2(|B_a|!).$$

The dimension adds across independent unresolved regions. A good algorithm should spend comparisons inside the blocks and none across blocks; the theorem says that, up to constants, a single global algorithm achieves exactly this behavior.

Merging two sorted lists. If P consists of two chains of lengths a and b, then sorting is merging. The number of compatible total orders is

$$e(P) = \binom{a+b}{a},$$

so

$$d_{\mathrm{sort}}(P) = \log_2 \binom{a+b}{a} = (a+b) H_2\left(\frac{a}{a+b}\right) + O(\log(a+b)),$$

where H_2 is the binary entropy function. The information-theoretic cost of merging is therefore another point on the same profile.

Clarifying resources and instances The partial-information theorem is clean because every object is visible. The instance is the public point P. The remaining state space is $\mathrm{Lin}(P)$. The resource is additional comparisons. The dimension is the logarithm of the remaining state space. The theorem proves that this dimension is both necessary and sufficient for computation in the comparison model.

This clarifies the role of representations. A DAG representation, a transitive closure, a transitive reduction, a graph-entropy computation, a balanced decision tree, or a heap trace can be very important for proving or implementing the upper bound. But the comparison dimension $\log e(P)$ does not depend on which representation is used. If one changes the resource from comparisons to total running time, the representation immediately becomes part of the model: reading a sparse DAG, querying a partial-order oracle, and preprocessing a closure are different computational tasks.

The value $e(P)$ is a complexity profile, not necessarily a quantity the algorithm must compute exactly. Counting linear extensions is #P-complete (Brightwell and Winkler, 1991). Thus the significance of the upper bound is not that an efficient algorithm first evaluates $e(P)$, but that its comparison behavior can be analyzed by $\log e(P)$. Proposition 4.3 separates the comparison-theoretic Bellman idea from computationally efficient pair selection; the efficient algorithms cited in Theorem 4.4 address the latter issue.

The theorem also separates pointwise analysis from neighboring notions. Average-case analysis integrates the profile over a distribution on public partial orders. Generic-case analysis asks for a bound on a large subset of public partial orders. Parameterized analysis upper-bounds the profile by width, number of incomparable pairs, poset dimension, or another chosen parameter. These are useful summaries. They are not the profile itself. For sorting under partial information, the intrinsic profile is exactly $P \mapsto \log e(P)$, up to constants.

6 Finite-resolution sorting of real keys

Partial orders are finite. Numerical sorting is not: the keys may be real numbers. The pointwise idea remains meaningful once the revealed information is finite-resolution.

Let $x = (x_1, \dots, x_n) \in [0,1)^n$, with ties broken by item index. Fix an integer $b \geq 0$ and reveal the bucket labels

$$q_b(i) := \lfloor 2^b x_i \rfloor \in \{0, \dots, 2^b - 1\}.$$

These labels induce an ordered-block partial order $P_b(x)$: every item in a smaller bucket must precede every item in a larger bucket, while items in the same bucket remain unresolved. Let

$$B_a(x,b) := \{i : q_b(i) = a\}, \qquad m_a(x,b) := |B_a(x,b)|.$$

Then

$$e(P_b(x)) = \prod_{a=0}^{2^b-1} m_a(x,b)!.$$

Theorem 6.1 (Finite-resolution pointwise sorting profile)**.** *After the b-bit bucket labels of the real keys are revealed, the optimal worst-case number of additional exact comparisons is*

$$C_b^\star(x) = \Theta\left(\sum_{a=0}^{2^b-1} \log_2(m_a(x,b)!) \right),$$

with the convention that the contribution of buckets of size 0 or 1 is zero. The upper bound is achieved by one uniform algorithm: bucket the items by their revealed labels and comparison-sort inside each nontrivial bucket.

Proof. The bucket labels impose a public ordered-block poset. A compatible exact total order is obtained by independently choosing the order inside each bucket, so the number of hidden completions is $\prod_a m_a!$. The decision-tree lower bound from Theorem 4.4 gives

$$C_b^\star(x) \geq \left\lceil \log_2 \prod_a m_a! \right\rceil = \left\lceil \sum_a \log_2(m_a!) \right\rceil.$$

For the upper bound, use merge sort inside every bucket and never compare elements from distinct buckets. Let $M(m)$ be the worst-case number of comparisons made by this fixed merge sort on m elements, with

$$M(0) = M(1) = 0, \qquad M(m) = M(\lfloor m/2 \rfloor) + M(\lceil m/2 \rceil) + m - 1 \quad (m \geq 2).$$

Then $M(m) \leq m\lceil \log_2 m \rceil$. Also $\log_2(m!) \geq (m/2)\log_2(m/2)$ for $m \geq 2$, after adjusting the absolute constant for $m = 2, 3$. Hence $M(m) \leq C \log_2(m!)$ for a universal constant C and all $m \geq 2$. Summing over buckets gives

$$\sum_a M(m_a) \leq C \sum_a \log_2(m_a!).$$

This proves the matching upper bound. □

The merge-sort upper bound also has an explicit Bellman potential. During a merge of two sorted lists of current lengths $r, s \geq 1$, take the potential to be $r + s - 1$. Comparing the two current first elements removes one element from one list, so the potential falls by exactly one in

both outcomes. During the recursive sorting phase on a block of size m, take the potential to be $M(m)$; splitting the block into $\lfloor m/2 \rfloor$ and $\lceil m/2 \rceil$ creates two recursive subproblems and a future merge budget of $m-1$, exactly matching the recurrence for $M(m)$. Summing this potential over all buckets gives a fully explicit Bellman proof of the upper bound in Theorem 6.1.

This theorem is the promised "beyond finite resolution" application inside sorting. With $b = 0$, all items lie in one bucket and the profile is $\log_2 n! = \Theta(n \log n)$. If b is large enough that every bucket contains at most one item, the revealed prefixes already determine the sorted order and no further comparisons are needed. At intermediate resolutions, the cost is exactly the entropy of the unresolved buckets, up to constants.

There is also a probabilistic reading. Suppose $X_1, \dots, X_n$ are i.i.d. from a continuous distribution on $[0, 1)$. Conditional on the bucket labels $(q_b(i))_{i=1}^n$, the relative order of the items inside each bucket is uniform, by exchangeability, and the buckets are already ordered. Therefore the conditional probability of the realized exact order is

$$\prod_a \frac{1}{m_a!},$$

and the conditional code length of the exact rank order is precisely

$$-\log_2 \Pr\{\mathrm{ord}(X) = \mathrm{ord}(x) \mid q_b(X) = q_b(x)\} = \sum_a \log_2(m_a!).$$

Thus the comparison profile, the finite-resolution code length, and the residual bucket entropy coincide in this example.

7 Connections with Occam, finite-scale dimension, and polytopes

The logarithmic profile should not be identified with only one neighboring theory. It has three typed readings. In a finite residual state space it is an Occam length. In finite-resolution numerical sorting it is a residual cell dimension, the discrete analogue of finite-scale pointwise statistical dimension. Through Stanley's order polytope it is also a polyhedral log-volume. These readings are useful precisely because they are kept separate. The Bellman potential is a fourth, algorithmic object: it is the dynamic-programming proof that the profile is attainable by one uniform comparison rule. We begin with the finite Occam specialization.

Let S be a finite nonempty set and let $\Delta(S)$ be the set of probability distributions on S. For $\mu \in \Delta(S)$, define the singleton pointwise code length

$$D_\mu(s) := \log_2 \frac{1}{\mu(s)}, \qquad s \in S,$$

with the convention $D_\mu(s) = +\infty$ if $\mu(s) = 0$. Then

$$\inf_{\mu \in \Delta(S)} \sup_{s \in S} D_\mu(s) = \log_2 |S|.$$

Indeed, for every μ, some point has mass at most $1/|S|$, while the uniform distribution attains equality. Applying this identity to $S = \mathrm{Lin}(P)$ gives

$$d_{\mathrm{sort}}(P) = \inf_{\mu \in \Delta(\mathrm{Lin}(P))} \sup_{\pi \in \mathrm{Lin}(P)} \log_2 \frac{1}{\mu(\pi)}.$$

Thus $\log e(P)$ is exactly the optimal worst-case Occam length of the finite set of hidden total orders still compatible with P (Blumer et al., 1987).

The same number has a polyhedral, and hence linear-programming, shadow. The partial order P defines Stanley's order polytope

$$\mathcal{O}(P) := \{x \in [0,1]^n : x_i \le x_j \text{ whenever } i \le_P j\}.$$

This is a feasible region cut out by linear inequalities. Its canonical triangulation has one simplex of volume $1/n!$ for each linear extension of P, and Stanley's formula gives

$$\mathrm{Vol}(\mathcal{O}(P)) = \frac{e(P)}{n!} \qquad \text{or equivalently} \qquad \log e(P) = \log n! + \log \mathrm{Vol}(\mathcal{O}(P)).$$

Thus $\log e(P)$ can also be read as the discrete log-volume of the remaining feasible ranking region (Stanley, 1986). This does not make partial-information sorting an algorithm for general linear programming. The sorting problem identifies a hidden ranking by adaptive comparison queries, whereas linear programming optimizes a linear objective over a polytope. But the analogy is useful: revealed inequalities shrink a feasible region, and the pointwise complexity measures the remaining combinatorial or volumetric size of that region.

There is an equivalent vertex view through the linear-ordering polytope. Encode a total order by pairwise variables $y_{ij} \in \{0,1\}$, where $y_{ij} = 1$ means that i precedes j. The public constraints $i <_P j$ fix the corresponding coordinates to one, and the remaining feasible ranking vertices are precisely the linear extensions of P. A comparison query asks for one still-unfixed coordinate of the hidden vertex. In this form, partial-information sorting identifies a hidden vertex of a public face of a ranking polytope.

The relation to pointwise statistical dimension is obtained by replacing finite singletons by finite-resolution metric balls. If S is equipped with the discrete metric $\rho(s,t) = \mathbf{1}\{s \neq t\}$, then for every $\varepsilon < 1$, the ball $B_\rho(s,\varepsilon)$ is the singleton $\{s\}$, and the finite identity above becomes

$$\inf_{\mu \in \Delta(S)} \sup_{s \in S} \log_2 \frac{1}{\mu(B_\rho(s,\varepsilon))} = \log_2 |S|.$$

The pointwise statistical dimension framework of Li and Xu (2026) starts from the same prior-mass template but removes the finite/discrete restriction. There the hypothesis class $\mathcal{F}$ may be uncountable, the distance ρ is a statistical semimetric, and the pointwise quantity has the form

$$D_\Pi(f,\varepsilon) := \log \frac{1}{\Pi(B_\rho(f,\varepsilon))}, \qquad B_\rho(f,\varepsilon) = \{g \in \mathcal{F} : \rho(f,g) \le \varepsilon\},$$

possibly with optimization or integration over scales. The ball is essential: under a continuous prior, $\Pi(\{f\}) = 0$ for typical f, so the singleton Occam length would be infinite. Finite-scale neighborhoods turn the question into: how much prior mass lies in a statistically indistinguishable neighborhood of this hypothesis? In smooth or representation-rich models, that mass is governed by local geometry, effective rank, and learned feature spectra rather than by finite cardinality.

Finite-resolution sorting is the exact bridge between the finite and uncountable pictures inside the sorting model. The real vector $x \in [0,1)^n$ is uncountable, but the revealed bucket cell

$$\{y \in [0,1)^n : q_b(y_i) = q_b(x_i) \text{ for all } i\}$$

induces a finite residual ordering problem. The pointwise complexity is not the code length of the singleton x, which would be infinite under continuous priors. It is the log-size of the finite set of rank orders still possible at resolution 2^{-b}:

$$\sum_a \log_2(m_a!).$$

Thus the citation to Li–Xu is right in a typed sense: their pointwise dimension motivates the finite-scale move from singleton states to neighborhoods or cells in uncountable spaces, while the theorem proved here remains a deterministic comparison-complexity theorem.

The mapping is therefore exact but typed:

finite sorting state space	$\mathrm{Lin}(P)$,
finite Occam envelope	$\log \lvert \mathrm{Lin}(P) \rvert$,
finite-resolution real-key cell	$\prod_a m_a!$ residual rankings,
polyhedral shadow	$\log n! + \log \mathrm{Vol}(\mathcal{O}(P))$,
discrete metric ball at $\varepsilon < 1$	$\{\pi\}$,
statistical analogue	$\log 1/\Pi(B_\rho(f, \varepsilon))$,
resource controlled here	additional comparisons,
resource controlled in learning	generalization or estimation error.

The shared principle is localization before taking the worst case. The mathematical objects and units are different. The present theorem is not a statistical generalization bound; it is a comparison-decision theorem. Conversely, the pointwise dimension of Li and Xu (2026); Lutz (2016) is not a sorting complexity; it is a finite-scale statistical complexity for hypothesis classes that may be uncountable.

The same distinction clarifies the relation with Bellman-sufficient information and computational complexity (Xu, 2026b). A code length can be assigned to the realized extension π conditional on P, and some extensions may have very short descriptions. Sorting under partial information asks for a minimax guarantee: one fixed comparison algorithm must be correct for every $\pi \in \mathrm{Lin}(P)$, and the profile takes the worst case over this compatible set. The operational length is therefore the minimax code length of the remaining set, $\log \lvert \mathrm{Lin}(P) \rvert$, not the description length of a lucky realized extension. The Bellman potential in Section 4 is the computational counterpart: it is a resource-to-go function whose one-step decrease proves a uniform worst-case upper bound, and in this note the resource-to-go function is the explicit log potential Φ_{KS}.

There is also a high-level connection with information-relaxation formulations of algorithmic robustness. For example, Xu (2026a) uses minimax language to compare what becomes possible when additional information is revealed to an algorithmic procedure. The present note studies the offline computational analogue in its cleanest finite form: reveal P, take the worst case over $\mathrm{Lin}(P)$, and compare a global algorithm with the benchmark that is allowed to know P. This shared quantifier pattern should not be confused with a shared formal framework. Here the resource is additional comparisons; in statistical or robustness problems the resource may be risk, loss, sample size, or an algorithm-dependent performance gap.

Thus pointwise does not mean nonuniform, and localized does not always mean geometric. In finite comparison problems, localization may reduce to an Occam-like finite state count. In finite-resolution numerical problems, it becomes the size of a residual cell. In uncountable statistical problems, localization requires metric balls at a chosen scale. In polyhedral problems, localization may appear as the volume or vertex structure of a feasible region. In all cases, the discipline is the same: fix the global rules first, and only then keep the pointwise profile instead of collapsing it immediately to its maximum.

8 Outlook

Sorting with partial information is a solved benchmark for pointwise worst-case complexity. The residual dimension $\log e(P)$ is explicit, the lower bound is elementary, and a logarithmic Bellman potential gives a transparent comparison upper bound. The finite-resolution real-key variant shows how the same pointwise logic moves from finite residual sets to uncountable inputs by working at a chosen scale. Efficient algorithms add a separate layer: they show that the favorable comparison profile can be attained without spending prohibitive time to discover the right comparisons.

More difficult problems have the same shape but are missing one of these ingredients. For shortest paths, one can count possible distance orderings of a fixed graph topology, but exact universal optimality depends on the data-structure model. For optimization, local curvature or error-bound constants often predict fast convergence, but the globally valid method and the cost of discovering favorable structure must be included. For learning, finite-scale pointwise dimensions can control risk, but their computational evaluation and the training-time profile are separate questions.

The moral is deliberately narrow and useful. First decide what information is legitimately revealed, and make that information the public instance. Then compare one global algorithm with the clairvoyant benchmark that knows this instance but not the hidden completion. When the global profile matches the benchmark point by point, worst-case complexity has a genuine pointwise refinement, and universal instance-optimality is achieved without abandoning uniform computation. Partial-information sorting shows this principle in finite form; finite-resolution sorting shows how the same principle begins to operate on uncountable numerical inputs.

Acknowledgements

This note is primarily a reformulation of the modern theory of sorting under partial information. It identifies that theory as an exact example of pointwise worst-case computational complexity; it views the log-potential argument as a deterministic-comparison instantiation of the Bellman-sufficient complexity viewpoint; and it adds finite-resolution sorting of real keys as a simple application. These statements are meant in the deterministic comparison model, not in a statistical or learning sense. The author used ChatGPT 5.5 Pro as a research and editorial assistant for generating technical suggestions and improving clarity of presentation.

References

Manuel Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967.

Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam's razor. *Information Processing Letters*, 24(6):377–380, 1987.

Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8:225–247, 1991.

Jean Cardinal, Samuel Fiorini, Gwenael Joret, Raphael M. Jungers, and J. Ian Munro. Sorting under partial information (without the ellipsoid algorithm). *Combinatorica*, 33:655–697, 2013.

Michael L. Fredman. How good is the information theory bound in sorting? *Theoretical Computer Science*, 1(4):355–361, 1976.

Bernhard Haeupler, Richard Hladik, John Iacono, Vaclav Rozhon, Robert E. Tarjan, and Jakub Tetek. Fast and simple sorting using partial information. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 3953–3973. SIAM, 2025.

Jeff Kahn and Jeong Han Kim. Entropy and sorting. *Journal of Computer and System Sciences*, 51(3):390–399, 1995.

Jeff Kahn and Michael E. Saks. Balancing poset extensions. *Order*, 1:113–126, 1984.

Shaojie Li and Yunbei Xu. Pointwise generalization in deep neural networks. arXiv:2605.18598, 2026.

Neil Lutz. A note on pointwise dimensions. arXiv:1612.05849, 2016.

Daniel Rutschmann. Sorting under partial information with optimal preprocessing time via unified bound heaps. arXiv:2604.12653, 2026.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423 and 27(4):623–656, 1948.

Richard P. Stanley. Two poset polytopes. *Discrete & Computational Geometry*, 1:9–23, 1986.

Ivor van der Hoog and Daniel Rutschmann. Tight bounds for sorting under partial information. In *Proceedings of the 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024.

Ivor van der Hoog, Eva Rotenberg, and Daniel Rutschmann. Simpler optimal sorting from a directed acyclic graph. In *Proceedings of the 2025 Symposium on Simplicity in Algorithms (SOSA)*, 2025.

Yunbei Xu. Pointwise complexity for Gaussian fields: Upper envelopes, algorithmic lower bounds, and separation. arXiv:2606.07931, 2026.

Yunbei Xu. Bellman-sufficient Information Complexity. arXiv:2606.11171, 2026.